

GreyNOC // Certified Pentest & Ethical Hacking

Getting Started with Public Bug Bounty

& Coordinated Vulnerability Disclosure

HackerOne · YesWeHack · Open Bug Bounty

A safety-first operator playbook for authorized, legal, repeatable testing.

GN-PUB-BBSTART-001

v1.1.2 · June 2026 · PUBLIC // AUTHORIZED DISTRIBUTION
grey-noc.com

Contents

About This Guide

4

1. Operator Standards Before Joining Any Platform

4

2. Legal Foundation: Authorization, Safe Harbor & Scope

5

Safe harbor language

5

security.txt: find the authorized channel

5

The good-faith standard

5

3. Set Up a Dedicated Operator Identity

6

4. Tooling & Passive Recon Baseline

7

Passive recon (always allowed, always first)

7

Active testing (only what the program permits)

7

5. HackerOne Setup

7

6. YesWeHack Setup

8

7. Open Bug Bounty

9

How this differs from HackerOne and YesWeHack

9

8. Choosing Your First Program

9

9. Understanding Severity: CVSS 4.0 & Impact

9

10. What Not to Report: Common Invalids & Duplicates

10

11. Worked Example 1 — Reflected XSS in a Search Field

11

12. Worked Example 2 — IDOR / Broken Access Control

12

13. Worked Example 3 — Open Redirect

13

14. Worked Example 4 — Cross-Site Request Forgery (CSRF)

14

15. Worked Example 5 — Sensitive Data / Misconfiguration Exposure

15

16. GreyNOC Operator Report Template

16

17. Triage Lifecycle & Communication Etiquette

17

18. Operator Automation Toolkit

18

Scope guard — fail safe, test nothing out of scope

19

security.txt — find the authorized channel politely

19

Report scaffold + submission tracker

20

19. Frontier AI for Bug Hunts

20

The slop problem (and why it matters to you)

20

Where frontier AI genuinely helps

20

Where it hurts — guard against these

21

The GreyNOC rule: human-in-the-loop accountability

21

Practical prompting for security work

21

20. First-Week Operator Training Path

21

21. Glossary of Terms

22

22. GreyNOC Public Commitment

23

About This Guide

This guide is for GreyNOC operators who want to participate in public bug bounty or coordinated vulnerability disclosure (CVD) programs in a professional, legal, and repeatable way. It takes a new operator from account creation through a clean first submission, and bakes in the safety-first discipline GreyNOC is known for.

Three rules sit above everything else in this document. They are not optional, and no program reward or finding justifies bending them:

- **Authorization first.** Only test systems where you have explicit, in-scope, written permission. The program page is your authorization boundary — not your assumptions.
- **Minimum impact.** Prove the vulnerability with the smallest possible footprint, then stop. Never access, copy, modify, or destroy real data.
- **Report privately.** Disclose through the approved channel only. Public disclosure happens when — and if — the platform and program allow it.

Read before you touch anything

Every platform and every program has its own scope, rules, reward terms, disclosure policy, and prohibited methods. Read the full program page before sending a single request to any target. When a rule is unclear, treat the action as out of scope and ask.

A note on terminology. A *bug bounty* program offers rewards (money, swag, or reputation) for valid findings. A *vulnerability disclosure program* (VDP) provides a legal, structured channel to report issues but usually offers no payment. *Coordinated disclosure* is the broader practice of reporting privately and giving the vendor time to fix before any public detail is released. Many beginners start on VDPs and CVD platforms precisely because the stakes and scope are easier to respect.

1. Operator Standards Before Joining Any Platform

GreyNOC operators follow these baseline rules on every platform, every program, every time:

- Test only in-scope assets.
- Do not scan, fuzz, brute force, spam, phish, social-engineer, or attempt denial-of-service unless the program explicitly allows it.
- Do not access, copy, modify, destroy, or disclose real user data.
- Stop testing the moment impact is proven.
- Report clearly and privately through the approved channel.
- Do not publicly disclose a finding unless the platform and program allow it.
- Keep clean notes: dates, URLs, parameters, steps, screenshots, request/response samples, and impact.

OWASP's vulnerability disclosure guidance reminds researchers to report responsibly, and warns that employment contracts can affect whether someone may report or personally claim a reward for vulnerabilities found using work resources. Operators follow GreyNOC policy and applicable client rules before claiming any reward personally.

GreyNOC house rule: separate the hat from the wallet

If you discovered a finding on GreyNOC time, on GreyNOC infrastructure, or under a client engagement, route the reward question through leadership *before* accepting a personal payout. Authorized testing and personal bounty hunting are different activities and should stay cleanly separated.

2. Legal Foundation: Authorization, Safe Harbor & Scope

The single biggest difference between ethical hacking and a crime is **authorization**. A bug bounty or disclosure program is the document that grants it. Before testing, confirm the program actually authorizes what you intend to do.

Safe harbor language

A program's *safe harbor* statement is its promise not to pursue legal action against good-faith researchers who stay within the published rules. Strong safe harbor language typically says the organization will not bring claims for authorized, good-faith research, will not refer you for prosecution, and considers your activity authorized under applicable computer-misuse law. The community-standard reference for this language is maintained by the disclose.io project.

- If a program has clear safe harbor: good. Stay inside the lines and you are covered as *described*.
- If a program has weak or missing safe harbor: treat scope even more conservatively, and when in doubt, do not test — ask first.
- Safe harbor protects in-scope, good-faith activity only. It does not cover out-of-scope testing, data exfiltration, or extortion.

security.txt: find the authorized channel

Many organizations publish a **security.txt** file (RFC 9116) that names their official reporting contact and policy. It is the polite, correct way to confirm where and how an organization wants to hear about issues. Check these locations before assuming there is no program:

```
https://example.com/.well-known/security.txt
https://example.com/security.txt
```

```
# Typical contents:
```

```
Contact: mailto:security@example.com
```

```
Policy: https://example.com/security-policy
```

```
Expires: 2027-01-01T00:00:00.000Z
```

```
Preferred-Languages: en
```

The good-faith standard

Operate as a reasonable, good-faith researcher at all times: act only to find and report weaknesses, avoid privacy violations and service disruption, stop at proof, and never use a finding for leverage, extortion, or personal gain outside the program's stated rewards. Good faith is not a loophole — it is the entire job.

Scope is a hard boundary, not a suggestion

“I was only looking” is not a defense if the asset was out of scope. Map in-scope and out-of-scope assets explicitly before you start, and re-check the scope each session — programs change their scope, exclusions, and rules over time.

3. Set Up a Dedicated Operator Identity

Before signing up anywhere, prepare a clean, professional research identity. A tidy profile builds triager trust and keeps your research activity separate from your personal accounts.

Recommended setup

- **Public handle:** a GreyNOC-compatible handle such as **greynoc-operatorname**, or a personal researcher handle approved by leadership.
- **Email:** use the approved GreyNOC operator email if you are reporting on behalf of GreyNOC.
- **MFA:** enable multi-factor authentication on every platform, no exceptions.
- **Bio:** short, accurate, professional.

Example profile bios**Example 1 — GreyNOC operator voice:****Bio sample A**

Security researcher and GreyNOC operator focused on responsible disclosure, web application security, and clear vulnerability reporting. I only test authorized assets and follow each program's scope and rules.

Example 2 — independent researcher voice:**Bio sample B**

Application security researcher. I specialize in access-control and input-handling flaws, write concise reproducible reports, and respect every program's scope, rate limits, and disclosure terms. Reachable via the platform; no DMs to staff.

Optional: an ORCID iD for researcher credibility

ORCID (Open Researcher and Contributor ID) is a free, persistent, name-independent identifier that distinguishes you from everyone with a similar name and gathers your body of work in one place. It is widely used in academic and scientific publishing; for a security researcher it offers a durable, verifiable identity you can attach to disclosed advisories, CVEs, conference talks, and write-ups as your reputation grows.

- **What it is:** a free 16-digit iD (e.g. **0000-0002-1825-0097**) from the non-profit ORCID registry; anyone may register, with no institutional affiliation required.
- **Why an operator might use it:** a single, persistent home for your published research and responsible-disclosure track record — handy when applying to private programs or citing prior work.
- **What it is NOT:** an identity and credibility tool, *not* a place to test systems or report vulnerabilities. Findings still go through the platforms in Sections 5–7.

Also prepare your kit

- Intercepting proxy — Burp Suite Community or OWASP ZAP.

- A separate, hardened test browser profile (no personal logins or extensions).
- Screenshot and screen-record tools for evidence.
- A notes template and a report template (see Section 13).
- A password manager.
- A VPN *only* if the program allows it — some programs ban or geofence VPN/Tor traffic.

Do not start with noisy automation

New operators begin with manual testing and careful observation. Automated scanners generate noise, trip rate limits, and frequently violate program rules. Earn the right to automate by first proving you can test cleanly by hand.

4. Tooling & Passive Recon Baseline

Before testing a single feature, understand the target the way a careful engineer would. Recon at the beginner stage is about *reading*, not attacking.

Passive recon (always allowed, always first)

- Read the entire program policy, scope, and exclusions — twice.
- Browse the application as a normal user. Map login, registration, search, upload, profile, password-reset, and any visible API endpoints.
- Note the technology stack from public signals (response headers, file extensions, framework fingerprints) without aggressive probing.
- Identify which features touch *your own* data so you can test access control safely against accounts you control.

Active testing (only what the program permits)

Move to active testing slowly and only within the rules. Respect rate limits. If the program forbids automated scanning, that includes most of the “scan everything” tooling — do it by hand.

Two registered accounts: your safest IDOR setup

For access-control testing, register two test accounts you fully control (call them Account A and Account B). This lets you check whether one account can reach another account's data *without ever touching a real user*. We use exactly this setup in Worked Example 2.

Recon discipline

Keep a running log from the first minute: timestamp, URL, what you did, what you observed. Good notes turn a lucky observation into a credible, reproducible report — and protect you by documenting that you stayed in scope.

5. HackerOne Setup

HackerOne is one of the largest bug bounty and vulnerability coordination platforms. Its help documentation directs new hackers to create an account and use the hacker dashboard / getting-started area to begin exploring platform resources.

Steps

1. Create a HackerOne account and verify your email.
2. Add your operator profile details.

3. Enable MFA.
4. Review the hacker dashboard.
5. Set payment preferences only if GreyNOC policy allows personal bounty payment.
6. Review reputation, signal, impact, and report-quality expectations.
7. Browse public programs and pick a beginner-friendly one with clear web scope.
8. Read the full policy before testing.

Confirm on every program before testing

- In-scope and out-of-scope domains.
- Eligible and excluded vulnerability types.
- Rate limits and testing restrictions.
- Disclosure rules and the reward table.
- Safe harbor language.
- Whether automated scanning is allowed.

What a good HackerOne report contains

- Clear title, affected asset, and vulnerability type.
- Severity estimate, summary, and exact steps to reproduce.
- Proof of concept, business/security impact, and suggested remediation.
- Screenshots or request/response evidence.
- Explicit confirmation that no real user data was accessed.

Disclosure on HackerOne is not automatic just because a report is submitted; the platform has a separate disclosure process. Check the current help-center guidance before requesting disclosure.

6. YesWeHack Setup

YesWeHack provides a hunter flow and recommends new hunters practice with *Dojo* challenges before moving into public programs. Dojo is a safe, hands-on practice environment; after completing challenges, hunters can move toward public programs.

Steps

1. Create a YesWeHack hunter account and verify your email.
2. Complete profile details and enable MFA.
3. Work through YesWeHack Dojo challenges.
4. Review public programs and read full scope and rules.
5. Start with low-risk manual testing.
6. Submit only complete, reproducible reports.

Current guidance: read the program description and rules carefully before testing, then submit a complete vulnerability report through the platform once you have identified a real issue. Report quality matters — complete, valid reports earn rewards and ranking points.

A good YesWeHack report

- Title, program name, asset, and vulnerability class.
- Summary, reproduction steps, and evidence.
- Impact, remediation, and notes about any limitations.

7. Open Bug Bounty

A note on “ORCID” (earlier mislabeled “ORIC”)

An earlier draft listed “ORIC” here as though it were a third platform. That was a typo for **ORCID** — the Open Researcher and Contributor ID. ORCID is a free researcher *identity* tool, not a bug bounty or disclosure platform, so it now lives where it belongs, under operator identity in Section 3. The coordinated-disclosure platform covered here is **Open Bug Bounty**. To verify any platform's name and disclosure model, use the community directory at **directory.disclose.io**.

Open Bug Bounty describes itself as a coordinated, responsible disclosure platform aligned with ISO/IEC 29147 guidelines. It lets researchers report vulnerabilities found *without* intrusive testing and submitted responsibly.

Steps

1. Create an Open Bug Bounty account and review platform rules.
2. Confirm the vulnerability type is accepted.
3. Avoid intrusive testing entirely.
4. Submit the vulnerability through the platform (you must be logged in).
5. Wait for validation and coordinated-disclosure handling.
6. Do not contact the organization aggressively or publish details outside the approved process.

How this differs from HackerOne and YesWeHack

HackerOne and YesWeHack usually center on formal programs with defined scopes and reward tables. Open Bug Bounty focuses on coordinated disclosure, including websites that may have no traditional paid program at all. That means operators must be **extra** careful: avoid intrusive testing, and never assume payment, permission for aggressive testing, or any right to public disclosure.

8. Choosing Your First Program

Look for

- Clear scope and a beginner-friendly web application.
- Public, well-written program rules with explicit safe harbor.
- A clear list of accepted vulnerability types.
- No production-sensitive areas (payments, healthcare, law enforcement, critical infrastructure).

Avoid as a first target

- Anything involving real customer data.
- Anything requiring authentication bypass, brute force, or destructive testing.
- Anything that requires heavy scanning, or anything with vague scope.

A good first goal is not “find a critical”

Your first goal is to submit **one clean, authorized, reproducible, professional report**. Process quality is the skill that compounds; criticals follow.

9. Understanding Severity: CVSS 4.0 & Impact

Most platforms ask for a severity estimate. The industry standard is the Common Vulnerability Scoring System (CVSS), currently **version 4.0** (published by FIRST in November 2023). CVSS produces a 0–10 score and a vector string; it measures *technical severity*, not your organizational risk.

Qualitative bands (consistent since CVSS v3)

Rating	Score range	Rough meaning for a first report
None	0.0	No security impact.
Low	0.1 – 3.9	Minor issue; limited or theoretical impact.
Medium	4.0 – 6.9	Real impact under some conditions (e.g. reflected XSS).
High	7.0 – 8.9	Serious impact; often pre-auth or sensitive data.
Critical	9.0 – 10.0	Severe, easily exploited, high blast radius.

CVSS 4.0 organizes metrics into four groups — **Base**, **Threat**, **Environmental**, and **Supplemental**. As a researcher you will usually provide the Base vector. A common honest beginner mistake is over-scoring: claim only what you actually proved.

Severity follows evidence, not ambition

Do not claim account takeover unless you safely demonstrated account takeover within scope. Inflated severity is the fastest way to lose triager trust. When unsure, score conservatively and let the triage team raise it.

Severity (how bad if exploited) is different from *likelihood* (how probable real-world exploitation is). Mature programs increasingly pair CVSS with EPSS for prioritization — useful context, though your job is to report the issue clearly, not to run their risk model.

10. What *Not* to Report: Common Invalids & Duplicates

Knowing what gets rejected saves everyone time and protects your reputation score. The following are commonly marked *informative*, *not applicable*, or out of scope unless the program explicitly says otherwise or you can demonstrate real, additional security impact:

- Missing security headers (HSTS, CSP, X-Frame-Options) with no demonstrated exploit.
- Self-XSS that requires the victim to paste a payload into their own console.
- Clickjacking on pages with no sensitive state-changing action.
- Open redirects with no further impact — *unless* the program accepts them (some do; see Worked Example 3).
- Verbose error messages or version banners with no exploit.
- Rate-limiting or CAPTCHA “weaknesses” with no concrete impact.
- Reports generated purely by an automated scanner with no manual validation.
- Vulnerabilities in third-party services that are out of the program's scope.

Duplicates and “informative” outcomes

If your report is marked **duplicate**, someone reported the same issue first — it does not mean your work was wrong. If it is marked **informative**, the team acknowledges it but will not act or reward it. Both are normal. Read the feedback, adjust, and move on. Reputation is built by signal (valid, well-scoped reports), not volume.

Turn rejections into a checklist

Before every submission, ask: did I demonstrate real impact, is it in scope, is it accepted by this program, and did I avoid the known-invalid list above? If yes to all four, submit with confidence.

11. Worked Example 1 — Reflected XSS in a Search Field

Training scenario

Fictional and instructional. Perform this only on an explicitly in-scope target where the program permits XSS testing and manual testing.

A program authorizes testing on **https://example-authorized-program.com**. The operator notices a search page that reflects the search term back into the page:

```
GET /search?q=test
```

Response (rendered):

```
Search results for: test
```

Step 1 — Confirm authorization

Before testing, confirm on the program page: the domain is in scope, reflected XSS is accepted, manual testing is allowed, no user data will be accessed, and public disclosure requires approval. If any are unclear, stop.

Step 2 — Harmless reflection test

Send a unique, harmless marker and confirm it is reflected:

```
GET /search?q=greynoc-test-12345
```

Response:

```
Search results for: greynoc-test-12345 <-- input is reflected
```

Step 3 — Check output encoding

Submit a benign HTML test string and observe how it is rendered:

```
GET /search?q=<b>greynoc-test-12345</b>
```

If the page shows the literal text `<b>greynoc-test-12345</b>` -> encoded (safe)

If the text renders in bold -> HTML injection

Step 4 — Prove execution safely

If the program allows XSS testing, use a minimal proof that only demonstrates script execution. Do not steal cookies, tokens, or session data; do not call external servers; do not persist payloads; do not affect other users.

```
# Minimal, self-contained proof of execution (alert-style only):  
GET /search?q=<script>alert('greynoc-xss-poc')</script>  
  
# Goal: show JS runs. Nothing is exfiltrated. Nothing persists.
```

Step 5 — Capture evidence

- Full affected URL and the HTTP request/response.
- Screenshot showing the alert firing.
- Date/time and scope confirmation.
- A note proving no real user data was accessed.

Step 6 — Determine impact (honestly)

Example impact: an attacker could craft a link that executes JavaScript in a victim's browser when opened, potentially enabling phishing or unauthorized actions if chained with other weaknesses. **Do not** claim account takeover unless you proved it safely and in scope.

Step 7 — Suggested remediation

Context-encode user-controlled output before inserting it into HTML, use framework-native escaping, avoid unsafe DOM insertion, and apply a Content Security Policy as defense-in-depth.

12. Worked Example 2 — IDOR / Broken Access Control

Training scenario

Fictional and instructional. Test access control only against accounts **you** control. Never view, alter, or exfiltrate another real user's data — that is the line between a valid report and a violation.

Insecure Direct Object Reference (IDOR) is a broken-access-control flaw where the application uses a user-supplied identifier to fetch a record without checking that the requester is allowed to see it. It is a great beginner target because you can prove it safely using two accounts you own.

Step 1 — Set up two controlled accounts

Register Account A and Account B (both yours). Log in as A and find an endpoint that fetches your own data by ID:

```
# Logged in as Account A:  
GET /api/invoices/1041 -> 200 OK (A's own invoice)  
  
# Note the ID is sequential and user-controllable.
```

Step 2 — Test the boundary with your OWN second account

Find Account B's object ID by browsing as B (in a separate browser profile). Then, as A, request B's ID and observe whether access control is enforced:

```
# B's own invoice id (discovered while logged in as B): 1042

# Now, still authenticated as Account A, request B's object:
GET /api/invoices/1042

Expected (secure):    403 Forbidden or 404 Not Found
Vulnerable (IDOR):   200 OK + returns Account B's invoice data
```

Step 3 — Stop at proof

If A can read B's record, you have demonstrated IDOR. **Stop immediately.** Do not iterate through other IDs, do not pull a range, and never touch an ID you cannot attribute to an account you control. One clean cross-account read between your own two accounts is sufficient proof.

Do not enumerate real users

Changing 1042 to 1043, 1044, 1045... to see “how many” you can read means accessing real users' data. That converts a clean finding into a serious violation. Prove the flaw with your own accounts, then report.

Step 4 — Evidence and impact

- Both request/response pairs (A reading A's object, then A reading B's object), with B's account being one you control.
- A clear note that both accounts are operator-controlled test accounts.
- Impact: any authenticated user can read (and possibly modify) other users' records by changing the ID — a confidentiality and potentially integrity breach.

Step 5 — Suggested remediation

Enforce object-level authorization on the server for every request: verify the authenticated user owns or is permitted to access the requested object before returning it. Do not rely on unguessable IDs alone; use access checks, and consider non-sequential identifiers as defense-in-depth.

13. Worked Example 3 — Open Redirect

Training scenario

Fictional and instructional. Only test on an in-scope target whose program accepts open-redirect reports. Many programs consider a bare open redirect low-impact or out of scope — check first (see Section 10).

An open redirect occurs when an application takes a URL or path from user-controlled input and redirects to it without validation. Attackers abuse it for convincing phishing links that begin on a trusted domain before bouncing to a malicious one.

Step 1 — Find a redirect parameter

Look for login, logout, SSO, or “continue” flows that carry a destination in the URL:

```
GET /login?next=/dashboard
GET /redirect?url=/account
GET /sso?returnTo=https://example-authorized-program.com/home
```

Step 2 — Test with a benign external destination

Substitute a harmless, clearly-yours destination and watch the response. Use a domain you control or a well-known safe placeholder — never a real phishing or malware site:

```
GET /redirect?url=https://greynoc-poc.example

Vulnerable response:
HTTP/1.1 302 Found
Location: https://greynoc-poc.example <-- redirects off-domain
```

If the app redirects to your external destination, the parameter is not validated. Also test common filter bypasses *only if the program allows it*, to show the validation is genuinely weak:

```
# Examples of patterns that defeat naive "starts with /" or "contains
# our-domain" checks (test only within scope):
url=/greynoc-poc.example           # protocol-relative
url=https://example.com.greynoc-poc.example # lookalike host
url=\/greynoc-poc.example          # backslash trick
```

Step 3 — Demonstrate impact honestly

Show that a victim clicking a link on the trusted domain lands on an attacker-chosen site. State the realistic impact — phishing / credential harvesting via a trusted-looking link — and note if it can be chained (e.g. with an OAuth flow) only if you actually demonstrated that chain.

Step 4 — Evidence

- The full request and the 302/redirect response showing the off-domain Location header.
- A screenshot of the browser landing on the external destination.
- Scope confirmation that open redirects are accepted by this program.

Step 5 — Suggested remediation

Validate redirect targets against an allow-list of known internal paths or hosts; prefer relative paths and reject absolute/protocol-relative URLs; map user input to server-side identifiers instead of accepting raw destinations.

14. Worked Example 4 — Cross-Site Request Forgery (CSRF)

Training scenario

Fictional and instructional. Test only on an in-scope target that accepts CSRF reports, and only against an account **you** control. Never trick a real user into performing an action.

CSRF abuses the fact that browsers automatically attach a user's session cookies to requests. If a state-changing action (change email, change settings, transfer something) does not require an unpredictable, request-specific token, an attacker can host a page that silently submits the action using the victim's logged-in session.

Step 1 — Find a state-changing request

Log in as your own test account and capture a request that changes account state — for example, updating the account email:

```
POST /account/email
Cookie: session=<your own session>
Content-Type: application/x-www-form-urlencoded

email=new@greynoc-poc.example
```

Step 2 — Check for an anti-CSRF token

Inspect the request and form. A protected endpoint includes an unpredictable token (e.g. **csrf_token**) tied to your session, and rejects the request if it is missing or wrong. If there is no token — or the server accepts the request when you remove it — the endpoint may be vulnerable.

```
# Remove or blank the token and replay the request as yourself:
POST /account/email      (csrf_token deleted)

Protected:  400 / 403  or  request rejected
Vulnerable: 200 OK    and the email actually changes
```

Step 3 — Build a minimal, self-contained proof

Demonstrate the issue with a tiny auto-submitting form that targets **your own** account. Do not deploy it anywhere a real user could reach it; the proof is that the action succeeds cross-site without a token.

```
<!-- greynoc-csrf-poc.html (run against your OWN test account only) -->
<form action="https://app.example.com/account/email" method="POST">
  <input name="email" value="new@greynoc-poc.example">
</form>
<script>document.forms[0].submit()</script>
```

Target only your own account

A valid CSRF proof changes *your* test account's state from a cross-site context. Never craft or send a payload that would execute against another real user — that crosses into attacking a person, not testing a control.

Step 4 — Evidence & impact

- The state-changing request with no/again-removed token, and the resulting success response.
- A short clip or screenshots of the PoC page changing your own account's email.
- Impact: an attacker could cause a logged-in victim's browser to perform this action without consent (e.g. hijack the account email, then trigger a password reset).

Step 5 — Suggested remediation

Require an unpredictable, per-session (ideally per-request) anti-CSRF token on every state-changing request and validate it server-side. Use **SameSite** cookies (Lax or Strict) as defense-in-depth, and prefer re-authentication for the most sensitive actions.

15. Worked Example 5 — Sensitive Data / Misconfiguration Exposure

Training scenario

Fictional and instructional. Confirming that a file is publicly reachable is in bounds on many programs; **downloading, using, or disclosing** real secrets is not. Stop at proof of exposure.

Security-misconfiguration findings are common beginner-friendly bugs: a build artifact, version-control directory, backup, or debug endpoint left publicly reachable. The skill here is proving exposure *without* harvesting whatever sensitive data is behind it.

Step 1 — Check for commonly-exposed paths (manually, in scope)

On an in-scope host, check a small, sensible set of paths by hand. This is not mass scanning — it is a handful of deliberate requests:

```
GET /.git/config      # exposed version-control metadata
GET /.env             # environment/config file
GET /backup.zip       # stray backup artifact
GET /server-status    # debug/status endpoint
```

Step 2 — Confirm exposure, then STOP

If one returns **200 OK** with content that should not be public, you have proven the misconfiguration. Capture just enough to show *what* is exposed — never page through it to extract secrets, credentials, keys, or personal data.

```
GET /.env -> 200 OK

# Proof goal: show the file is reachable and is a real config file.
# Do NOT copy, use, or disclose any secret value it contains.
# Redact secrets in your evidence (e.g. DB_PASSWORD=****REDACTED****).
```

Exposure proof, not data theft

The valid finding is “this sensitive resource is publicly reachable.” Reading, copying, or using the secrets inside it converts a clean report into unauthorized access. Redact everything sensitive in your evidence.

Step 3 — Evidence & impact

- The full URL and the 200 response headers showing the resource is served publicly.
- A redacted snippet that proves the file type without revealing secret values.
- Impact: depending on the file, this can expose credentials, internal structure, source code, or customer data — a confidentiality breach and often a stepping stone to deeper compromise.

Step 4 — Suggested remediation

Remove the artifact from the web root, block access to version-control and dotfiles at the server/CDN layer, disable debug/status endpoints in production, and rotate any credentials that were exposed. Add a deployment check so build artifacts and secrets never ship to public paths.

16. GreyNOC Operator Report Template

Use this structure for HackerOne, YesWeHack, or coordinated disclosure. Fill every field; delete nothing.


```

Title:          [Vuln Type] in [Feature/Endpoint] allows [Impact]
Summary:        2-4 sentences describing the issue plainly.
Program:
Platform:
Affected asset / Endpoint(URL):
Vulnerability type:
Severity estimate (CVSS 4.0 vector if known):
Scope confirmation:  In-scope? Type accepted? Method allowed?
Testing performed:   Manual / what was and was NOT done.

Steps to reproduce:
  1.
  2.
  3.

Proof of concept:   (no cookie/token/data theft; minimal proof only)
Observed result:
Expected result:
Impact:             (claim only what was proven)

Evidence:
  Screenshot:
  Request:
  Response:
  Timestamp:

Remediation recommendation:

Operator safety notes:
- No real user data accessed.
- No destructive testing performed.
- No persistence attempted.
- No denial-of-service testing performed.
- Testing stayed within program scope.

Disclosure status:  Private until program approval.

```

Example titles (good vs vague)

Vague (avoid)	Specific (use)
"XSS found"	Reflected XSS in q parameter on /search
"Access bug"	IDOR in /api/invoices/{id} exposes other users' invoices
"Redirect issue"	Open redirect in url parameter on /redirect enables phishing
"CSRF maybe"	CSRF on /account/email allows attacker-forced email change
"File exposed"	Public /.env on app.example.com exposes service credentials

17. Triage Lifecycle & Communication Etiquette

After you submit, the report moves through a lifecycle. Understanding it keeps you patient and professional.

State	What it means	Your move
New / Submitted	Awaiting triage review.	Wait. Do not re-test or nudge.
Triaged	Validated as a real issue.	Answer questions; stay available.
Needs more info	Triager needs detail.	Reply precisely and promptly.
Duplicate	Reported earlier by someone else.	Note it; move on.
Informative	Acknowledged, no action/reward.	Learn from feedback.
Resolved	Fixed by the vendor.	Confirm if asked; await disclosure terms.

Communication rules

- Be patient; triage takes time. Do not message employees outside the platform.
- Answer triage questions clearly and quickly — responsiveness builds reputation.
- Do not re-test aggressively after submitting.
- Never threaten disclosure or imply leverage. That is extortion, not research.
- Do not disclose publicly unless and until the program approves it.

One conversation, one channel

Keep all communication inside the platform's report thread. Off-platform contact with staff, social-media call-outs, or "I'll go public" pressure can void safe harbor and end your standing on the platform.

18. Operator Automation Toolkit

Automation in bug hunting should make you **more disciplined**, not more reckless. The right things to automate are scope safety, finding the authorized channel, evidence hygiene, report scaffolding, and submission tracking — never discovery at volume. GreyNOC ships a small, dependency-free Python toolkit (companion to this guide) that does exactly this.

What this toolkit does NOT do

It does not scan, fuzz, brute force, or exploit. Many programs prohibit automated scanning outright. These are workflow and hygiene tools that support careful manual testing — keep a human in the loop for every decision and every submission.

Script	Purpose	Network
scope_guard.py	Confirm a target is in scope before you test it.	No
securitytxt.py	Find the authorized reporting channel (RFC 9116).	1-2 GETs
evidence_log.py	Append timestamped UTC evidence + captures.	No
report_gen.py	Scaffold a complete GreyNOC report in Markdown.	No
tracker.py	Track submissions and lifecycle state locally.	No
cvss_band.py	Map a CVSS base score to its qualitative band.	No

Scope guard — fail safe, test nothing out of scope

Reads a JSON scope file and refuses anything not explicitly in scope. It makes no requests; its only job is to stop you straying.

```
$ python3 scope_guard.py --scope scope.json \
    --target https://app.example.com/search

Program : example-public-program
Target   : app.example.com
Verdict  : IN SCOPE -> proceed within the program rules.
Reminder: automated scanning is NOT allowed for this program.
```

Core logic — out-of-scope is checked first, and anything unlisted is treated as out of scope:

```
def host_of(target):
    if '://' not in target:
        target = 'https://' + target
    return (urlparse(target).hostname or '').lower().strip('.')

def matches(host, patterns):
    return any(fnmatch.fnmatch(host, p.lower()) for p in patterns)

if matches(host, out_scope): return 'OUT OF SCOPE -> STOP'
if matches(host, in_scope):  return 'IN SCOPE -> proceed'
return 'NOT LISTED -> treat as OUT OF SCOPE until confirmed'
```

security.txt — find the authorized channel politely

At most two timed GET requests with a clear User-Agent; fails safe to “no channel found” rather than guessing.

```
$ python3 securitytxt.py example.com

[FOUND] https://example.com/.well-known/security.txt
Contact: mailto:security@example.com
Policy:  https://example.com/security-policy
Expires: 2027-01-01T00:00:00.000Z
```

Report scaffold + submission tracker

Generate a complete, GreyNOC-structured report skeleton, then track it through the lifecycle — locally, no network:

```
$ python3 report_gen.py --title 'Reflected XSS in q on /search' \
  --asset 'https://app.example.com/search' --type 'Reflected XSS' \
  --program example-public --platform HackerOne --severity Medium
[WRITE] reports/reflected-xss-in-q-on-search.md

$ python3 tracker.py add --id H1-1001 --program example-public \
  --title 'Reflected XSS in q on /search'
$ python3 tracker.py set-status --id H1-1001 --status triaged
$ python3 tracker.py list
H1-1001 triaged example-public Reflected XSS in q on /search
```

Evidence hygiene

`evidence_log.py` appends a UTC-timestamped JSONL entry per observation and can copy a saved request/response capture alongside it — turning loose notes into a clean, defensible audit trail that also proves you stayed in scope.

19. Frontier AI for Bug Hunts

Frontier AI is an accomplice, not an autopilot. Used well, it compresses the boring parts of research and sharpens your reports. Used carelessly, it produces confident, professional-looking nonsense that wastes triagers' time and burns your reputation.

The slop problem (and why it matters to you)

Through 2025 and into 2026, bug bounty programs were flooded with “AI slop” — LLM-generated reports that read convincingly but describe vulnerabilities that do not exist, cite functions that aren't real, and can't be reproduced. The curl project reported that roughly a fifth of its 2025 submissions were AI slop while genuine ones fell to about one in twenty, and it paused its long-running bug bounty in early 2026 before resuming once report quality recovered. Its maintainer's bar is blunt: if you cannot explain a bug in a few sentences and reproduce it, do not submit it. Platforms adapted too — HackerOne framed 2025 as the rise of the “bionic hacker,” with valid AI-assisted findings up sharply year over year and an AI assistant (its Hai agent) now pre-checking reports for scope and completeness before a human reviews them.

The same stretch proved the flip side. AI wielded by skilled humans found real, novel vulnerabilities — including genuine CVEs in curl and a batch of previously unknown OpenSSL issues — that careful operators responsibly disclosed. In short, AI collapsed the *median* report quality while raising the *ceiling*. Which side of that split you land on is a choice you make on every single submission.

Submitting unverified AI output is self-harm

Triagers can spot AI slop, and false or low-effort reports cost you reputation — which limits how many reports you can file and your access to private programs. If you cannot personally reproduce and explain a finding, it is not ready to submit. Full stop.

Where frontier AI genuinely helps

- **Understanding fast.** Explaining unfamiliar code, frameworks, error messages, or an obscure protocol so you can test intelligently.
- **Idea generation.** Brainstorming test cases and edge conditions for a feature — which you then verify by hand.
- **Reading the rules.** Summarizing long program policies so you don't miss a scope or exclusion clause (then confirm against the source).
- **Report craft.** Tightening your *own* verified findings into clear, well-structured reports and remediation advice.
- **Safe tooling.** Writing workflow/hygiene scripts like the toolkit in Section 18 — scope checks, loggers, trackers.

Where it hurts — guard against these

- **Hallucinated findings.** Models invent plausible-sounding vulns, CVEs, endpoints, and parameters. Never trust an AI-asserted bug you haven't reproduced.
- **False confidence on severity.** AI will happily call a low-impact issue “critical.” Score from evidence, not from the model.
- **Scope blindness.** The model doesn't know the program's scope or authorization. That judgment stays human, every time.
- **Privacy leaks.** Never paste real user data, secrets, tokens, or another person's information into any AI tool.

The GreyNOC rule: human-in-the-loop accountability

Emerging community norms (including from open-source security working groups) converge on a simple standard that GreyNOC adopts in full:

1. A human must review, understand, and be able to explain every piece of AI-assisted content before it is used or submitted.
2. Reproduce the finding yourself, manually, before reporting it.
3. Disclose substantial AI assistance when a program asks.
4. No autonomous agents acting against targets, opening PRs, or submitting reports on their own.
5. The quality bar does not move because AI was involved — you remain fully responsible for what you submit.

Practical prompting for security work

- Give the model context and your *own* observations; ask it to explain or critique, not to invent findings.
- Ask it to argue against your hypothesis — “why might this NOT be exploitable?” catches false positives early.
- Keep a personal, iterated prompt library for recurring tasks (recon summaries, report polishing, remediation write-ups).
- Treat every factual claim (a CVE, an API behavior, a config default) as unverified until you confirm it against primary sources or a live test.

Accomplice, not autopilot

Let frontier AI carry the cognitive overhead — reading, summarizing, explaining, drafting — while you keep ownership of authorization, verification, and judgment. That division of labor is how AI makes you a faster, sharper, and *more* trustworthy operator.

20. First-Week Operator Training Path

Day 1 — Accounts & rules

- Create platform accounts, enable MFA, complete profiles, read platform rules.

Day 2 — Practice labs

- Complete YesWeHack Dojo or equivalent labs; practice writing reports from lab findings.

Day 3 — Program study

- Review three public programs; compare scope, rewards, accepted bugs, and exclusions; pick one beginner-friendly target.

Day 4 — Passive recon

- Passive recon only; map visible features (login, search, upload, profile, password reset, API).

Day 5 — Manual test one feature

- Test a single feature by hand; look for reflected input, access-control mistakes, unsafe redirects, exposed debug info, or business-logic flaws.

Day 6 — Draft a report

- Write a full draft report even if you do not submit; review for clarity, evidence, and scope.

Day 7 — Submit or document

- Submit one valid, low-risk, in-scope report if the finding is real. If not, document lessons learned and move to another approved feature.

21. Glossary of Terms

Term	Meaning
CVD	Coordinated Vulnerability Disclosure — reporting privately and giving the vendor time to fix.
VDP	Vulnerability Disclosure Program — a legal reporting channel, usually without payment.
Safe harbor	A program's promise not to pursue good-faith, in-scope researchers legally.
Scope	The exact assets, methods, and vuln types a program authorizes.
PoC	Proof of Concept — the minimal demonstration that a vulnerability is real.
IDOR	Insecure Direct Object Reference — broken object-level access control.
XSS	Cross-Site Scripting — injecting script that runs in a victim's browser.
CVSS	Common Vulnerability Scoring System — standard 0-10 severity scoring (v4.0 current).
EPSS	Exploit Prediction Scoring System — likelihood of real-world exploitation.
Triage	The program's review that validates and rates a submitted report.
Signal	A reputation measure rewarding valid, high-quality reports over noise.
Duplicate	A valid issue someone else reported first.
Informative	Acknowledged report that will not be acted on or rewarded.
security.txt	RFC 9116 file naming an organization's security contact and policy.
CSRF	Cross-Site Request Forgery — forcing a logged-in user's browser to perform an action.
Misconfiguration	A security weakness from insecure defaults or exposed resources, not a code bug.
AI slop	Convincing-looking but false or unreproducible LLM-generated vulnerability reports.
Human-in-the-loop	A person reviews, understands, and is accountable for all AI-assisted output.

22. GreyNOC Public Commitment

GreyNOC operators participate in vulnerability disclosure with a safety-first mindset. Our goal is to help organizations identify and fix security weaknesses through authorized testing, clear reporting, and responsible communication. We do not perform intrusive testing, access user data, or disclose vulnerability details outside approved channels.

Pre-submission checklist

- ✓ Asset is in scope and the vuln type is accepted.
- ✓ I proved real impact with the minimum footprint, then stopped.
- ✓ No real user data was accessed, modified, or exfiltrated.
- ✓ No prohibited methods (scan/brute/DoS/social) were used.
- ✓ Report is complete, reproducible, and honestly scored.
- ✓ Disclosure stays private until the program approves it.

References for current rules: each platform's official help center; FIRST CVSS v4.0 (first.org/cvss); OWASP vulnerability disclosure guidance; RFC 9116 (security.txt); the disclose.io directory and safe-harbor project; and reporting from HackerOne, the curl project, and the OpenSSF on AI-assisted submissions and responsible AI use. The companion GreyNOC Operator Automation Toolkit ships alongside this guide. Platform specifics change — always confirm against the live program page before testing.

GN-PUB-BBSTART-001 · v1.1.2 · June 2026 · Brand template GN-TPL-PUB-001 · PUBLIC // AUTHORIZED DISTRIBUTION · grey-noc.com